모델 서빙 및 AIOps 구성

수업 운영 가이드

AI 서비스를 위한 SW 기초 Full-stack Engineering · 3일(24시간) · 이론 50% / 실습 50%

수업 진행 일정

- 이번 과목은 **가상 종목 HumanAI Corporation(HAIC)의 일별 시세로 다음날 종가를 예측하는 LSTM 모델** 하나를 Day1(서빙) → Day2(MLOps) → Day3(AIOps) 순서로 **하나의 `serving_app/` 위에 쌓아 올리는** 방식으로 진행합니다.
- 실습 코드는 **완성형이 아니라 스켈레톤**으로 제공됩니다. 업로드·학습·대시보드 같은 배관 코드는 완성되어 있고, **각 Day 의 핵심 학습 목표에 해당하는 부분만 TODO 로 비워 두었습니다.** 학생은 TODO 를 채우며 원리를 체득합니다.
- 평가는 **개인 실습(서브노트 제출)**과 **조별 실습(조별 미니 프로젝트)** 두 가지로 나누어 진행합니다.
  - → **[개인 실습] 서브노트** : 매일 개념 실습과 종합 실습을 스스로 수행하며 겪은 시행착오, TODO 구현 과정, 실행 결과(로그·캡처)를 서브노트에 기록합니다. Day1~Day3 내용을 누적해 하나의 서브노트로 각 반별 슬랙 쓰레드에 제출합니다.
  - → **[조별 실습] 미니 프로젝트** : 팀이 선정한 AI 서비스(예측 모델)를 대상으로, 개인 실습에서 익힌 서빙 → 버전 관리·배포 → 모니터링 → 드리프트 자동 대응 파이프라인을 팀 도메인에 맞게 치환·통합하고 기획서와 동작 시연으로 발표합니다.
- 모든 날의 종합 실습은 **전날 산출물 위에 이어서** 진행합니다. Day1 서버를 Day2 에서 MLflow·Docker 로 감싸고, Day2 컨테이너 앱에 Day3 모니터링·재학습을 붙입니다. 따라서 전날 TODO 를 끝내지 못하면 다음 날 실습이 막히므로, 각 Day 종료 시 완료 기준을 반드시 확인합니다.
- 완성된 전체 기능(4 탭 대시보드, 운영 지표, 알람 등)은 별도 **데모 패키지(`project_answer_demo/`)**로 시연만 하고, 학생 실습은 핵심 루프(업로드 → 학습/서빙 → 드리프트 감지 → 재학습)에 집중합니다.

  MLflow 서버, Prometheus·Grafana 같은 별도 모니터링 스택, Kubernetes 배포는 이번 실습 범위에서 제외합니다. MLflow도 별도 서버 없이 로컬 sqlite로 동작하고, 컨테이너는 단일 컨테이너로 구성합니다. 도구를 많이 쓰는 것보다 "서빙 → 배포 → 모니터링 → 자동 대응"이 한 줄로 이어지는 원리를 체득하는 것이 이 과목의 목표입니다.

상세 일정

시간	Day 1 : 모델 서빙	Day 2 : MLOps	Day 3 : AIOps
09:00–11:50	모델 서빙·AIOps 이해 (AI 서비스에서의 위상과 가치) 모델 서빙 아키텍처 (Training vs Serving, 배치 vs 실시간, REST/gRPC, 동기/비동기) FastAPI 기초 (라우팅, Pydantic 스키마, Swagger)	Day1 회고 리마인드 MLOps 개요 (DevOps와의 차이, ML 라이프사이클, 성숙도) 모델 버전 관리 및 레지스트리 연동 (MLflow Tracking/Registry) CI/CD for ML (컨테이너화의 역할, 환경 일관성)	Day2 회고 리마인드 AIOps 개요 (MLOps와의 차이, 모니터링 3대 대상) 모델 모니터링 (지연시간·처리량·에러율, 로그, 알림) 모델 드리프트 & 자동 대응 (데이터/컨셉 드리프트, 재학습 트리거)
11:50–13:10	점심	점심	점심
13:10–15:00	비동기 처리 (async/await, batching) 서빙 안정성 (입력 검증, 에러 핸들링, 헬스체크, Lazy/Eager 로딩)	Dockerize (Day1 서빙 앱 이미지 빌드·실행) 데이터/모델 검증 (스키마 검증, 성능 회귀 테스트, 배포 게이트)	드리프트 감지 시뮬레이션 (정상/드리프트 배치 주입) 모델 서빙 및 AIOps 파이프라인 가이드 (1~3일 회고)
15:00–16:00	[개념 실습] FastAPI 추론 엔드포인트 (스켈레톤 기동, /docs, /predict·/health 확인)	[개념 실습] MLOps 라이프사이클 매핑 (오늘 실습이 어느 단계인지 한 장 요약)	[개념 실습] AIOps 설계 (지표·로그 구조, 탐지 → 알림 → 재학습 트리거 구조 작성)
16:00–18:00	[종합 실습] 추론 서버 빌드업 : 업로드 → baseline 학습 → 입력 검증·에러 핸들링·헬스체크 → Lazy vs Eager 측정 서브노트 Day1 작성	[종합 실습] MLflow 학습·게이트·Production 승격 → `_load_from_mlflow()` 구현 → Docker 빌드·실행 조 편성 + 팀 아이디어 킥오프 서브노트 Day2 작성	[종합 실습] 드리프트 감지·자동 재학습 TODO 완성 → aiops.log 검증 조별 미니 프로젝트 통합·데모 서브노트 최종 제출

조별 미니 프로젝트 발표는 Day3 종료 시 또는 반별로 지정한 일정에 진행합니다. 반 상황에 따라 시간 배분은 조정할 수 있습니다.

평가 연계 : 무엇을 보여주면 되는가

종합 실습 평가는 개인 서브노트 2개 항목과 조별 미니 프로젝트 2개 항목으로 이루어집니다. 아래 표는 각 항목이 **어떤 관점에서 보는지**와 **서브노트·기획서를 어느 방향으로 작성하면 되는지**를 정리한 것입니다.

구분	평가 관점 (배점)	작성 방향
개인 1-1	서빙·MLOps·AIOps 운영 원리 이해 및 가치 연결 (60)	서브노트 **1. 이해관계자 가치 · 2. AI 솔루션 · 핵심 질문 1)~5)**가 중심입니다. 개념 정의를 옮겨 적기보다, 직접 측정한 수치(시작·응답 시간, RMSE 등)를 근거로 답하고 그 결과가 어떤 이해관계자에게 어떤 가치를 주는지까지 연결해 보세요.
개인 1-2	서빙~AIOps 파이프라인 구현 및 실행 증빙 (40)	서브노트 **3. 아키텍처 구성도 · 4. 트러블 슈팅 · 5. 레퍼런스**가 중심입니다. 구성도는 업로드부터 재배포까지 흐름이 끊기지 않게, 트러블슈팅은 "오류 메시지 → 원인 → 해결 명령 → 해결 전후 결과" 형식으로, 증빙은 아래 "Day별 필수 증빙"을 실제 실행 화면·로그로 남기세요.
조별 2-1	문제 정의 및 운영 시나리오 설계 (40)	기획서 ① Pain Point ② AI 솔루션·운영 목표 ③ 운영 설계 (배포 게이트·임계값·드리프트 대응 정책) (본문 8번)
조별 2-2	서빙~AIOps 파이프라인 구현 및 발표 완성도 (60)	기획서 ④ 아키텍처 구성도 ⑤ API 명세 ⑥ 동작 화면 스냅샷 (본문 8번)

개인 서브노트는 "실제로 구현·검증했는가(1-1)"와 "왜 그렇게 운영하는가를 이해했는가(1-2)" 두 방향을 모두 보여줄 때 완성됩니다. 어느 한쪽만 두껍게 쓰기보다 두 방향이 서로를 뒷받침하도록 작성하세요.

실습 서브 노트 (개인, Day1~Day3 누적)

서브노트는 아래 1~5번 순서로 작성합니다. **1·2번과 핵심 질문**은 "왜 그렇게 운영하는가"(평가 1-2)를, **3·4·5번**은 "실제로 구현·검증했는가"(평가 1-1)를 보여주는 부분입니다.

1. 이해관계자 가치 (Pain Point) : 모델을 학습만 하고 서빙·운영 체계가 없다면 AI 서비스 이해관계자(사용자, 운영자, 모델 개발자, 경영진)가 실제로 어떤 불편을 겪는지 정의합니다. 예를 들어 모델이 노트북 안에만 있어 서비스에서 쓸 수 없는 문제, 새 모델을 배포했더니 성능이 오히려 떨어진 문제, 시장 환경이 바뀌었는데 아무도 모르고 틀린 예측을 계속 내보내는 문제가 왜 중요한지 먼저 설명합니다.

2. 이를 해결하기 위한 AI 솔루션 : 정의한 Pain Point를 서빙(FastAPI) · MLOps(버전 관리·게이트·컨테이너) · AIOps(모니터링·드리프트 감지·자동 재학습)가 각각 어떻게 해결하는지, 핵심 기능과 그 안에서 각 기술이 담당하는 역할을 설명합니다.

3. 아키텍처 구성도 : CSV 업로드 → baseline 학습(scaler 고정) → MLflow 학습·게이트·Production 승격 → FastAPI 서빙(Lazy/Eager) → Docker 컨테이너 → 예측 기록 누적 → RMSE 드리프트 판정 → aiops.log 알림 → fine-tuning 재학습 → 게이트 재검증 → 재배포까지의 흐름이 한눈에 보이도록 구성합니다.

특히 다음 질문에 대한 답이 제시되어야 합니다. 일반론보다 **본인이 직접 측정·관찰한 값**을 근거로 답하세요.

1) Lazy vs Eager : 두 방식의 서버 시작 시간과 첫 요청 응답 시간을 직접 측정한 결과는? 실제 서비스라면 어떤 상황에서 어떤 방식을 택하겠는가?

2) 학습 시점과 서빙 시점의 일치 : 왜 `/predict`는 단일 값이 아니라 20거래일(SEQ_LEN) 시퀀스를 받는가? 왜 `scaler.pkl`은 Day1에 한 번만 fit하고 Day2·Day3에는 다시 fit하지 않는가?

3) 배포 게이트 : RMSE $4.00 게이트가 없다면 어떤 일이 생기는가? 게이트를 통과하지 못한 버전은 어떻게 처리되며, 그것이 운영자에게 주는 가치는 무엇인가?

4) 드리프트 판정 기준 : 판정 윈도우(21건)와 임계값($4.00)을 더 짧게/길게, 더 낮게/높게 잡으면 오탐과 미탐은 어떻게 달라지는가? 정상 배치와 드리프트 배치의 실제 RMSE는 얼마였는가?

5) 재학습 방식 : 드리프트 감지 후 왜 처음부터(scratch) 학습하지 않고 Production 가중치에서 이어서(warm start) fine-tuning하는가? 재학습된 모델이 게이트를 통과하지 못하면 서비스는 어떻게 되는가?

4. 트러블 슈팅 : TODO를 구현하며 겪은 오류(오류 메시지 → 원인 → 해결 명령 → 해결 전후 결과)를 기록합니다. 새로운 용어에 대한 이해나 요소 기술 간 연결(FastAPI ↔ MLflow ↔ Docker ↔ 로그)에 대한 정리도 좋습니다. (부록 1 참고)

5. 레퍼런스 (필요한 스냅샷) : 교수자 강의 내용을 키워드 중심으로 요약하고, 아래 "Day별 필수 증빙"에 해당하는 실행 화면·로그를 첨부합니다. 수업에서 이해한 내용이 실제로 구현·검증되었다는 증빙 자료 역할을 합니다.

Day 별 필수 증빙 (서브노트 5 번 레퍼런스에 첨부)

Day	서브노트에 첨부할 증빙
Day1	`/docs`(Swagger) 화면과 `/predict` Try it out 요청·응답(`predicted_close`, `model_version`)

Day	서브노트에 첨부할 증빙
	잘못된 입력(시퀀스 19개, close ≤ 0 등)에 대한 422 검증 오류 응답 `/health` 응답(`model_loaded`, `loading_mode`)의 Lazy/Eager 차이 Lazy vs Eager 측정표 : 서버 시작 시간, 첫 요청 응답 시간, 두 번째 요청 응답 시간 `train_baseline_v1.py` 실행 로그(baseline v1 RMSE)
Day2	`train_and_register.py` 실행 로그(`[GATE PASSED]` 또는 `[GATE FAILED]`)와 MLflow에 기록된 params·metric(rmse) `_load_from_mlflow()` 구현 코드와 `MODEL_SOURCE=mlflow` 기동 후 `/predict` 응답(`model_version: production`) `docker compose up --build` 빌드·실행 로그, 컨테이너(`localhost:8000/docs`)에서의 Swagger 동작 화면 오늘 실습을 ML 라이프사이클 단계에 매핑한 한 장 요약
Day3	`compute_rmse()`·`batch_test()`·`check_and_trigger()`·`send_batch()` 구현 코드 `simulate_drift.py` 실행 결과 : 정상 배치와 드리프트 배치의 `drift_check`(RMSE 포함) `logs/aiops.log`에 `[WARN] drift detected` → `[INFO] retrain triggered` → `[OK] new_rmse=...` 순서로 쌓인 화면(대시보드 재학습 로그 패널) 재학습 후 `/predict` 응답 확인(새 Production 버전 반영 여부)

개인 실습 / 조별 미니 프로젝트 기준

개인 실습은 제공된 HAIC 스켈레톤의 TODO 5개를 직접 채워, **업로드 → 학습/서빙 → 드리프트 감지 → 재학습**의 핵심 루프가 본인 PC에서 끝까지 동작하게 만드는 것이 목표입니다. 도메인은 바꾸지 않고 HAIC 그대로 진행합니다.

조별 미니 프로젝트는 개인 실습으로 확보한 파이프라인을 **팀이 정한 AI 서비스 도메인으로 치환**하는 것이 목표입니다. 예측 대상·입력 시퀀스·배포 게이트 지표·드리프트 신호·재학습 정책을 팀 도메인에 맞게 다시 정의하고(본문 7번 도메인 매핑표), 그 결과를 기획서와 동작 시연으로 발표합니다. 팀원 개개인의 역할(기획, 서빙 API, 배포, 모니터링)은 달라도 되지만, 결과물은 하나의 파이프라인으로 이어져야 합니다.

실습 배포 패키지 (전달 항목)

항목	내용
실습 스켈레톤	**project.zip** : `serving_app/`(FastAPI 앱, 라우터, 모니터링, Dockerfile, docker-

항목	내용
	compose.yml, 대시보드), `data/`(features.py, storage.py, 예시 CSV), `scripts/`(baseline 학습, 드리프트 시뮬레이션), `requirements.txt`, `README.md`
예시 데이터	`data/sample_haic_prices.csv` : 756거래일(약 3년), `Date, Open, High, Low, Close, Volume`. 실제 IBM 2007~2009 시세(상승 → 금융위기 최대 -45% 폭락 → 회복)를 참조해 날짜만 재구성한 가상 HAIC 데이터
데모 패키지	`project_answer_demo/` : 4탭 대시보드, 운영 지표, 알람 등 완성 기능 시연용 (교수자 시연, 학생 실습 대상 아님)
학생 수정 대상	TODO 5개 : `model_loader._load_from_mlflow()`, `routers/predict.batch_test()`, `monitoring/drift_detector.compute_rmse()`, `monitoring/retrain_trigger.check_and_trigger()`, `scripts/simulate_drift.send_batch()`
수정하지 않는 항목	`routers/data.py`(업로드), `routers/logs.py`(로그 조회), `static/index.html`(대시보드) : 업로드·결과 확인용 배관 코드로 완성형 제공

실습 시, 개인 PC 공통 실행 절차

실습에서는 **서빙 → MLOps → AIOps를 하나의 serving_app 위에 쌓는 스켈레톤**을 제공합니다.

모든 명령은 **project 폴더(프로젝트 루트)**에서 실행합니다. `serving_app/` 안에서 실행하면 import 경로가 어긋납니다.

0. 가상환경 구성 및 패키지 설치 (Python 3.11 권장, TensorFlow 포함이라 설치에 수 분 소요)

```
python3.11 -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

1. [Day1] 서버 기동 (Lazy 모드 기본값이라 데이터·모델 없이도 서버가 뜹니다)

```
uvicorn serving_app.main:app --host 0.0.0.0 --port 8000
# 대시보드 http://localhost:8000/  ,  API 문서 http://localhost:8000/docs
```

2. [Day1] 데이터 업로드 : 대시보드 업로드 카드에서 data/sample_haic_prices.csv 업로드 (또는)

```
curl -F "file=@data/sample_haic_prices.csv" http://localhost:8000/data/upload
```

3. [Day1] baseline 모델·스케일러 생성 (별도 터미널, 최초 1회)

```
python scripts/train_baseline_v1.py      # -> serving_app/models/haic_v1.keras, scaler.pkl
```

4. [Day1] Eager 모드와 비교 (서버 재기동 후 시작 시간·첫 요청 시간 측정)

```
LOADING_MODE=eager uvicorn serving_app.main:app --host 0.0.0.0 --port 8000
```

5. [Day2] MLflow 학습 → 게이트 → Production 승격

```
python serving_app/train_and_register.py
MODEL_SOURCE=mlflow uvicorn serving_app.main:app --host 0.0.0.0 --port 8000   #
_load_from_mlflow() 구현 후
```

6. [Day2] 컨테이너로 재현 (단일 컨테이너, 이미지 빌드 시 학습·등록까지 완료되는 자기 완결형 이미지)

```
docker compose -f serving_app/docker-compose.yml up --build
# 또는: docker build -f serving_app/Dockerfile -t haic-serving:v2 . && docker run -p
8000:8000 haic-serving:v2
```

7. [Day3] 드리프트 시뮬레이션 (서버가 떠 있는 상태에서 별도 터미널)

```
python scripts/simulate_drift.py        # 정상 배치 -> 드리프트 배치 순서로 /predict/batch-test
에 주입
```

주의 : README 일부 예시는 `--port 8077`로 되어 있지만, `simulate_drift.py`(API_URL)와 Dockerfile은 **8000** 포트를 사용합니다. 실습에서는 **8000으로 통일**하세요. 8077을 쓰려면 `simulate_drift.py`의 `API_URL`도 함께 바꿔야 합니다.

Windows PowerShell에서는 환경변수를 `$env:LOADING_MODE="eager"; uvicorn ...` 형식으로 지정합니다.

제공되는 API 계약 정보 확인

서버를 띄운 뒤 `http://localhost:8077/docs`(Swagger UI)에서 아래 엔드포인트를 모두 Try it out으로 호출해 볼 수 있습니다.

Method	URL	Day	역할 / Request → Response
GET	/	공통	실습용 대시보드 (업로드, 드리프트 시뮬레이션, 재학습 파이프라인, 재학습 로그)
POST	/predict	Day1	`{"sequence": [{"close": 160.0, "volume": 1200000}, ... 20개]}` → `{"predicted_close": 161.23, "model_version": "v1-local"}`
GET	/health	Day1	`{"status": "ok", "model_loaded": true/false, "loading_mode": "lazy"}`
POST	/data/upload	Day1~	multipart CSV 업로드 (Date·Close·Volume 필수, 최소 41행) → `{"filename": "haic_17...csv", "rows": 756}`
GET	/data/status	Day1~	최신 업로드 파일의 행 수, 기간, 종가 최소·최대
POST	/predict/batch-test	Day3	`{"prices": [종가 41개 이상]}` → `{"predictions": [...], "drift_check": {"status": "ok" | "retrain_triggered", ...}}`

Method	URL	Day	역할 / Request → Response
GET	/logs, /logs/{파일명}	Day3	`logs/aiops.log` 목록·내용 조회 (재학습 이력 확인용, 읽기 전용)

대시보드는 상대 경로로 API를 호출하므로 어떤 포트로 띄워도 동작하지만, 스크립트(simulate_drift.py)는 고정 URL을 쓰므로 포트를 맞춰야 합니다.

각 반별 체크리스트

- ☐ 각 반에서 개인별 project.zip 전달 및 Python 3.11 가상환경 + `pip install -r requirements.txt` 완료 확인 (TensorFlow 설치 실패 PC 사전 파악)
- ☐ Docker Desktop 설치·실행 확인 (Day2 전까지) 및 이미지 빌드 가능 디스크 여유(수 GB) 확인
- ☐ Day1 : 업로드 → baseline 생성 → `/predict` 200 응답 확인, Lazy/Eager 측정표 작성 확인
- ☐ Day2 : `[GATE PASSED]` 로그와 `MODEL_SOURCE=mlflow` 응답 확인 후 Docker 빌드 진행 (TODO 미완성 상태로 빌드하면 컨테이너 기동 실패)
- ☐ Day2 종료 시 조 편성 및 팀 도메인(예측 대상) 킥오프
- ☐ Day3 : `aiops.log` 에 WARN → INFO → OK 3 단계 로그가 쌓이는지 확인, 재학습 후 모델 반영 여부 확인
- ☐ 서브노트 최종 제출(반별 슬랙 쓰레드) 및 조별 미니 프로젝트 발표·전체 회고 진행

모델 서빙 & AIOps 실습 가이드

무엇을 이해하고, 무엇은 몰라도 되는가?

1. 왜 서빙·MLOps·AIOps 를 함께 배우는가?

지금까지 여러분은 주로 "모델을 잘 학습시키는 방법"을 배웠습니다. 그런데 실무에서 AI 프로젝트가 실패하는 지점은 학습보다 그 이후인 경우가 훨씬 많습니다. 노트북(Jupyter) 안에서 RMSE가 잘 나온 모델도, 다른 서비스가 호출할 수 있는 형태로 제공되지 않으면 아무 가치를 만들지 못합니다. 이것이 **모델 서빙**이 필요한 이유입니다. Day1에서 FastAPI로 모델을 `/predict` API로 감싸는 순간, 모델은 "파일"에서 "서비스"가 됩니다.

서비스가 되고 나면 곧 두 번째 문제가 생깁니다. 모델은 한 번 만들고 끝나는 것이 아니라 계속 새 버전이 나옵니다. 어떤 데이터로, 어떤 설정으로 학습했고, 성능이 얼마였는지 기록하지 않으면 "지금 서비스 중인 모델이 무엇인지"조차 알 수 없고, 더 나쁜 모델이 배포되어도 막을 방법이 없습니다. **MLOps**(Day2)는 학습 기록(MLflow Tracking), 버전 관리(Model Registry), 배포 게이트(RMSE $4.00), 컨테이너화(Docker)로 이 문제를 해결합니다.

마지막으로, 배포된 모델은 시간이 지나면 조용히 나빠집니다. HAIC 데이터의 2008년 금융위기처럼 시장 환경이 바뀌면, 코드에는 아무 오류가 없는데도 예측이 계속 틀립니다. 일반 소프트웨어는 버그가 나면 에러가 나지만, AI 서비스는 **에러 없이 틀립니다. AIOps**(Day3)는 예측 오차를 지켜보다가(RMSE 모니터링) 드리프트를 감지하면 알림을 남기고, 재학습·재검증·재배포까지 자동으로 이어지게 만듭니다.

> *정리하면, 서빙은 "모델을 쓸 수 있게" 만들고, MLOps는 "모델을 믿고 바꿀 수 있게" 만들며, AIOps는 "모델이 나빠져도 스스로 회복하게" 만듭니다. 이번 실습은 이 세 단계를 하나의 serving_app 위에 순서대로 쌓아, AI 서비스가 운영되는 전체 생애주기를 직접 경험하는 과정입니다.*

2. 실무에서는 이렇게 활용된다 (성숙도의 진화 흐름)

많은 조직이 처음부터 완전 자동화된 파이프라인을 갖추지 않습니다. 보통은 데이터 과학자가 노트북에서 학습한 모델 파일을 개발자에게 전달하고, 개발자가 수동으로 서버에 올리는 단계(MLOps 성숙도 Level 0)에서 시작합니다. 이 단계에서는 "누가, 언제, 어떤 모델을 올렸는지" 추적이 어렵고, 성능이 떨어져도 사용자 불만이 들어오기 전까지 아무도 모릅니다.

서비스가 커지면 학습 기록과 버전 관리, 배포 전 자동 검증(게이트)이 붙고(Level 1), 이어서 운영 지표 모니터링과 드리프트 감지에 따른 자동 재학습까지 연결됩니다(Level 2). 이번 실습의 Day1 → Day2 → Day3은 정확히 이 진화 순서를 따릅니다. 실무에서는 MLflow 대신 SageMaker·Vertex AI 같은 관리형 서비스를, 로그 파일 대신 Prometheus·Grafana·Slack 알림을 쓰는 경우가 많지만, **"기록 → 게이트 → 배포 → 감시 → 자**

동 대응"이라는 구조 자체는 동일합니다.

3. 시작하기 전에 꼭 알아야 할 것

이번 실습의 목표는 LSTM 모델을 더 정확하게 만드는 것이 아닙니다. 모델 아키텍처(LSTM 3층), 데이터 전처리(시퀀스·스케일러), 업로드·로그 조회·대시보드는 모두 완성된 형태로 제공됩니다. 여러분이 채울 것은 **각 Day의 핵심 원리가 드러나는 TODO 5개**뿐입니다.

실무에서도 MLOps·AIOps 엔지니어가 모델 자체를 설계하는 경우는 드뭅니다. 데이터 과학자가 만든 모델을 받아 "안정적으로 서빙하고, 안전하게 교체하고, 나빠지면 알아채는" 체계를 만드는 것이 역할입니다. 모델 내부는 블랙박스로 두고 입력·출력·성능 지표만 다루는 연습을 한다고 생각하세요.

> *LSTM의 게이트 수식이나 TensorFlow 내부 동작을 이해하려고 시간을 쓰는 것은 이번 실습 범위를 벗어납니다. 모델은 "(20, 2) 시퀀스를 넣으면 다음날 종가 하나를 돌려주는 함수"로만 이해해도 충분합니다.*

4. 이해해야 할 것 vs 몰라도 되는 것

구성요소	알아야 할 것	몰라도 되는 것
FastAPI 서빙 (main.py, routers/, schemas.py)	라우터 등록, Pydantic 스키마가 입력을 검증하는 방식(길이 20, close > 0), 422 응답의 의미, StaticFiles를 마지막에 mount하는 이유	Starlette·uvicorn 내부 구조, ASGI 스펙
모델 로딩 (model_loader.py)	Lazy(첫 요청 시 로드 + 캐시)와 Eager(시작 시 로드)의 차이와 트레이드오프, MODEL_SOURCE로 local ↔ mlflow 전환되는 "조립 블록" 구조	Keras 모델 직렬화 포맷(.keras) 내부
LSTM 모델·피처 (lstm_model.py, data/features.py)	입력 (20, 2) → 출력 1, 시간 순서를 지킨 train/test 분할, scaler.pkl을 Day1~3 내내 재사용하는 이유	LSTM 게이트 수식, 하이퍼파라미터 튜닝
MLflow (train_and_register.py)	run·param·metric 기록, Model Registry 등록, 게이트 통과 시 Production 승격, `models:/HAIC_Predictor/Production` URI로 로드	백엔드 스토어 구조, MLflow 서버 배포·인증
Docker (Dockerfile, docker-compose.yml)	각 줄의 의미(베이스 이미지, 의존성 설치, 소스 복사, 빌드 시 학습·등록, ENV, EXPOSE, CMD), 포트 매핑 8077:8077	이미지 레이어 최적화, 멀티스테이지 빌드, 오케스트레이션

구성요소	알아야 할 것	몰라도 되는 것
모니터링·드리프트 (monitoring/, predict.batch_test)	슬라이딩 윈도우 21건 RMSE vs 임계값 $4.00, 판정 → 알림 로그 → fine-tuning → 게이트 재검증 흐름, aiops.log 3단계 로그	KS 검정·PSI 등 통계적 드리프트 검정 (조별 프로젝트 확장 과제로는 가능)
대시보드·업로드·로그 조회 (static/, data.py, logs.py)	사용법 : 업로드하면 최신 파일이 학습에 쓰이고, 로그 패널에서 재학습 증거를 본다는 것	index.html 내부 JavaScript, 업로드 파일 저장 방식

이 표가 핵심입니다. "몰라도 되는 것" 칸의 내용을 이해하려다 막혔다면, 그것은 여러분의 문제가 아니라 실습 범위를 벗어난 질문을 스스로에게 던진 것입니다.

5. 파일 지도와 TODO 목록

세 Day의 산출물은 독립적이지 않고 하나의 `serving_app/` 위에 쌓입니다. `model_loader.py`가 Day1 로컬 모델 → Day2 MLflow Production 모델로 전환되는 지점이 그 연결고리입니다.

Day	파일 · 함수	해야 할 일
Day1	(TODO 없음) main.py, schemas.py, model_loader.py, routers/predict.py·health.py	구조 이해, 업로드 → baseline 생성 → /predict 호출, Lazy/Eager 측
Day2	`model_loader._load_from_mlflow()`	`mlflow.tensorflow.load_model("models:/HAIC_Predictor/P` 로 로드하고, 스케일러는 로컬 scaler.pkl에서 로드해 LoadedModel
Day3	`drift_detector.compute_rmse()`	RMSE = sqrt(mean((actual − predicted)²)) 직접 구현, 빈 리스트면
Day3	`routers/predict.batch_test()`	prices를 길이 20 슬라이딩 윈도우로 잘라 예측, {predicted, actual} recent_predictions에 누적하고 최근 21건 유지
Day3	`retrain_trigger.check_and_trigger()`	최신 업로드 CSV의 최근 (21 + 20)행으로 `fine_tune(rows)` 호출, 여부에 따라 [OK] 로그와 결과 반환
Day3	`scripts/simulate_drift.send_batch()`	`requests.post(API_URL, json={"prices": ...})`로 배치 전송 drift_check 출력

각 TODO 위치의 주석에 힌트 코드가 들어 있습니다. 주석을 그대로 풀어 쓰는 데서 멈추지 말고, "이 줄이 없으면 무슨 일이 생기는가"를 서브노트에 한 줄씩 남기면 핵심 질문 답변(평가 1-2)의 근거가 되고, 구현 중 겪은 오류는 트러블슈팅(평가 1-1)으로 기록하면 됩니다.

6. Day 별 실습 상세

Day1 : HAIC baseline LSTM 을 FastAPI 로 서빙

1. 서버를 Lazy 모드(기본값)로 기동합니다. 데이터·모델 파일이 없어도 서버는 뜨지만, 이 상태에서 `/predict` 를 호출하면 모델 파일이 없어 실패합니다. 이것이 Lazy 방식의 "늦게 드러나는 오류"입니다.
2. 대시보드에서 `sample_haic_prices.csv` 를 업로드하고 `/data/status` 로 확인합니다.
3. `python scripts/train_baseline_v1.py` 로 baseline 모델(`haic_v1.keras`)과 스케일러(`scaler.pkl`)를 만듭니다. 이 스케일러는 Day3 까지 절대 다시 fit 하지 않습니다.
4. Swagger 에서 `/predict` 를 호출합니다. 20 개 시퀀스, 19 개 시퀀스, close = 0 등으로 바꿔 보며 200 과 422 응답을 비교합니다.
5. Lazy 와 Eager 각각으로 재기동하여 서버 시작 시간, 첫 요청 응답 시간, 두 번째 요청 응답 시간을 측정해 표로 정리합니다. (서버 콘솔의 `[lazy] model loaded in ...s` / `[eager] ...` 로그를 활용)

Day2 : MLflow 학습·레지스트리·컨테이너화

1. `python serving_app/train_and_register.py` 로 base 학습(100 epoch, SEED 42 고정)을 실행합니다. RMSE 가 게이트($4.00) 이하면 `HAIC_Predictor` 로 등록되고 Production 으로 승격됩니다.
2. `_load_from_mlflow()` TODO 를 구현하고 `MODEL_SOURCE=mlflow` 로 서버를 띄워 `model_version` 이 `production` 으로 바뀌는지 확인합니다. `main.py` 와 `train_and_register.py` 는 건드리지 않고 이 파일 하나만 바꿔 소스가 전환되는 "조립 블록" 구조를 확인합니다.
3. `docker compose -f serving_app/docker-compose.yml up --build` 로 컨테이너를 빌드·실행합니다. 이미지 빌드 시점에 샘플 데이터 시드 → baseline → MLflow 학습·등록이 모두 끝나는 자기 완결형 이미지이므로, `_load_from_mlflow()`가 완성되어 있어야 컨테이너가 정상 기동합니다.
4. 오늘 실습(학습 기록 → 게이트 → 등록 → 컨테이너 배포)이 ML 라이프사이클의 어느 단계에 해당하는지 한 장으로 매핑합니다.

Day3 : 드리프트 감지·자동 재학습 부착

1. `compute_rmse()` → `batch_test()` → `check_and_trigger()` → `send_batch()` 순서로 TODO 를 채웁니다. (의존 순서대로 채워야 중간 확인이 가능합니다)

2. `python scripts/simulate_drift.py` 를 실행합니다. 정상 배치(일별 변동성 1.2% 랜덤워크)와 드리프트 배치(변동성 3 배)가 각각 41 개 종가로 전송되고, 배치마다 21 건의 (predicted, actual) 쌍이 쌓여 바로 판정됩니다.
3. 정상 배치는 `status: ok`, 드리프트 배치는 `status: retrain_triggered` 가 나오는지 확인합니다. 랜덤워크이므로 실행마다 RMSE 가 달라집니다. 여러 번 실행해 결과 분포를 서브노트에 기록하면 좋습니다.
4. 대시보드 재학습 로그 패널(또는 `logs/aiops.log`)에서 `[WARN] drift detected` → `[INFO] retrain triggered` → `[OK] new_rmse=... production promoted` 순서를 확인합니다.
5. 재학습 후 `/predict` 가 새 Production 모델로 응답하는지 확인합니다. 서빙 중인 모델 캐시가 자동으로 갱신되지 않는 점에 유의하세요(부록 1 의 6 번). 이 문제를 어떻게 해결했는지가 좋은 트러블슈팅 기록이 됩니다.

7. 팀 아이디어로 치환하는 법 (도메인 매핑표)

HAIC 스켈레톤은 "시계열 예측 모델을 서빙하고 운영하는" 구조입니다. 개념 구조만 같으면 어떤 예측 서비스로도 치환할 수 있습니다. 다음 예시는 B2B 관점(기업의 운영 담당자에게 가치를 제공)으로 이해 바랍니다.

템플릿 개념 (HAIC)	예시 : 프랜차이즈 수요 예측	우리 팀 아이디어
예측 대상 : 다음날 종가	다음날 매장별 판매량	
입력 시퀀스 : 최근 20거래일 (close, volume)	최근 28일 (판매량, 프로모션 여부)	
데이터 공급 : 대시보드 CSV 업로드	POS 일마감 데이터 업로드	
배포 게이트 : RMSE ≤ $4.00	MAPE ≤ 15% (발주 오차 허용 범위)	
드리프트 신호 : 변동성 3배 급변	신메뉴 출시, 휴가철, 경쟁점 오픈	
재학습 : 최근 21거래일 fine-tuning	최근 4주 데이터로 fine-tuning	
알림 : aiops.log [WARN]	본사 SCM 담당자 메신저 알림	
이해관계자 : 투자 분석가, 운영자	점주(재고 폐기), 본사 SCM(발주),	

템플릿 개념 (HAIC)	예시 : 프랜차이즈 수요 예측	우리 팀 아이디어
	경영진(손실)	

팀별로 이 표를 먼저 채우고 시작하세요. 표가 채워지면 스키마(입력 형식), 게이트 기준, 드리프트 임계값, API 명세가 거의 자동으로 정해집니다. 실제 데이터가 없다면 HAIC처럼 공개 데이터를 참조해 같은 컬럼 형식의 가상 CSV를 만들어도 됩니다.

8. 조별 미니 프로젝트 기획서에 포함해야 할 내용

발표 기획서는 아래 여섯 가지 흐름으로 구성해 주세요. 앞 단계의 결과가 다음 단계의 근거가 되도록 순서대로 이어져야 합니다. (①~③은 평가항목 2-1, ④~⑥은 평가항목 2-2)

① **이해관계자 가치 (Pain Point)** : 팀이 정의한 이해관계자가 겪는 불편함과 그 중요성, 그리고 서비스 장애·응답 지연·모델 품질 저하가 생겼을 때 이해관계자에게 미치는 영향을 정의합니다. (7번 도메인 매핑표 기준)

② **AI 솔루션과 운영 목표** : Pain Point를 해결하는 핵심 기능, 서빙되는 모델의 역할, 그리고 운영 요구사항(예 : 응답 시간 1초 이내, 가용성, 품질 목표 RMSE/MAPE)을 제시합니다.

③ **운영 설계** : 배포 게이트 기준(입력 스키마 검증, 성능 회귀 테스트, 임계값), 모니터링 지표와 임계값, 드리프트 발생 시 대응 정책(알림 → 재학습 → 게이트 재검증 → 실패 시 기존 버전 유지 또는 롤백)을 근거와 함께 설계합니다.

④ **아키텍처 구성도** : 데이터 업로드, 학습·MLflow 레지스트리, FastAPI 서빙(컨테이너), 예측 기록·로그, 드리프트 감지, 알림·재학습 트리거의 흐름을 팀 서비스명으로 치환해 화살표로 표시합니다.

⑤ **API 명세** : 추론·헬스체크·업로드·배치 테스트·로그 조회 등 실제 호출되는 엔드포인트 목록(Method, URL, Request/Response 예시, 422 등 에러 응답)을 정리합니다. Swagger UI에서 확인한 내용을 옮기면 됩니다.

⑥ **동작 화면 스냅샷** : 모델 버전 전환 전/후 응답, 컨테이너 실행 화면, 드리프트 주입 → 알림 → 재학습 트리거 로그 등 요청 전/후 상태 변화가 드러나는 실제 동작 화면을 캡처·시연합니다.

> *"왜(Pain Point) → 무엇을(솔루션·운영 목표) → 어떻게 지킬 것인가(운영 설계) → 어떻게 구현했나(아키텍처·API) → 결과(동작 화면)"의 순서를 지키면, 발표를 듣는 사람도 팀의 논리를 자연스럽게 따라올 수 있습니다.*

9. 실습 진행 순서 (완료 기준 체크리스트)

- ☐ `/data/upload` 로 CSV 를 올리면 업로드 완료로 표시되는가 (`/data/status` 로도 확인)
- ☐ `/predict` 가 200 으로 응답하고, 잘못된 입력에는 422 로 거부하는가
- ☐ Lazy vs Eager 의 시작 시간·첫 요청 시간 차이를 수치로 설명할 수 있는가
- ☐ `[GATE PASSED]` 후 `MODEL_SOURCE=mlflow` 에서 `model_version: production` 으로 응답하는가
- ☐ 컨테이너(`localhost:8000/docs`)에서도 동일하게 동작하는가
- ☐ 정상 데이터로는 RMSE $4 이내, 드리프트 데이터로는 $4 초과가 재현되는가
- ☐ `logs/aiops.log` 에 `[WARN] drift detected` → `[INFO] retrain triggered` → `[OK] new_rmse=...` 순서로 기록되는가
- ☐ 재배포 후 `/predict` 호출 시 새 Production 버전이 응답하는가

10. 막혔을 때 질문하는 법

"MLflow가 왜 이렇게 동작하나요?"처럼 도구 내부를 묻기보다, "`MODEL_SOURCE=mlflow`로 띄웠더니 `/predict`에서 이런 오류 메시지가 나옵니다. `train_and_register.py` 로그는 `[GATE PASSED]`였습니다"처럼 **실행한 명령, 오류 메시지, 직전 단계의 결과**를 함께 알려 주세요.

> *질문 범위를 좁히면 훨씬 빠르게 답을 얻을 수 있습니다. 먼저 부록 1의 자주 겪는 오류 표를 확인하고, 해당하지 않으면 질문해 주세요.*

부록 1. 자주 겪는 오류와 해결 (트러블슈팅)

#	증상	원인	해결
1	`simulate_drift.py` 실행 시 Connection refused	서버를 8000 포트로 띄웠는데 스크립트의 API_URL은 8077	서버를 `--port 8077`으로 띄우거나 `API_URL`을 실제 포트로 수정
2	`/predict` 500 오류, 로그에 haic_v1.keras 없음	Lazy 모드라 서버는 떴지만 baseline 모델을 아직 만들지 않음	CSV 업로드 후 `python scripts/train_baseline_v1.py` 실행
3	FileNotFoundError : 업로드된 HAIC 데이터가 없습니다	`data/uploads/`가 비어 있음 (학습·시뮬레이션은 항상 최신 업로드 파일 사용)	대시보드 또는 `/data/upload`로 CSV 업로드. 프로젝트 루트에서 실행했는지도 확인
4	`/predict` 422	sequence가 20개가 아니거나	Pydantic 스키마가 정상 동작한 것. 오류 응

#	증상	원인	해결
	Unprocessable Entity	close ≤ 0, volume < 0	답의 `loc·msg`를 읽고 입력 수정 (서브노트 증빙으로 활용)
5	`MODEL_SOURCE=mlflow` 기동 시 NotImplementedError / 컨테이너가 바로 종료됨	`_load_from_mlflow()` TODO 미구현. Dockerfile은 `MODEL_SOURCE=mlflow`, `LOADING_MODE=eager`로 시작하므로 기동 즉시 실패	TODO 구현 후 재빌드 (`--build`)
6	재학습 로그는 [OK]인데 `/predict` 결과·버전이 그대로	`model_loader._model_cache`에 예전 모델이 남아 있음 (재학습 후 캐시를 비우지 않음)	promoted일 때 `model_loader._model_cache = None`으로 캐시를 비우거나 다시 로드하도록 보완. 버전 번호를 응답에 넣으면 확인이 쉬움
7	재학습 직후 다음 배치에서 또 드리프트 판정	`recent_predictions`에 재학습 전 예측이 그대로 남아 있음	재학습 후 윈도우를 비우는(초기화) 정책을 설계해 적용하고 그 이유를 기록
8	드리프트 배치 요청이 오래 걸리거나 타임아웃	`check_and_trigger()` 안에서 fine-tuning이 요청 처리 중에 동기적으로 실행됨	Day1의 비동기 처리와 연결 : FastAPI BackgroundTasks 등으로 재학습을 응답과 분리하는 개선 가능
9	`[GATE FAILED] rmse=... > 4.0`, 이후 mlflow 로드 실패	게이트 미통과로 Production 버전이 없음 (환경에 따라 RMSE가 달라질 수 있음)	기존 Production 유지가 정상 동작임을 이해하고, 원인(데이터·epoch)을 기록. 필요 시 재실행
10	MLflow에서 stage 관련 FutureWarning	MLflow 3에서 Model Registry stage 방식이 deprecated (동작은 함)	실습은 그대로 진행. 조별 확장 과제로 alias(`models:/HAIC_Predictor@champion`) 방식 전환 가능
11	docker build가 매우 오래 걸림 / 용량 부족	TensorFlow 포함 이미지(1GB대) + 빌드 시 학습까지 수행	최초 1회는 수 분 소요가 정상. 디스크 여유 확보, 재빌드 시 캐시 활용

Dockerfile 주석의 `routers/metrics.py`, `scripts.aggregate_metrics` 언급은 데모 패키지 기준 설명입니다. 스켈레톤에는 해당 파일이 없으며 빌드에는 영향이 없습니다.

부록 2. 자주 묻는 질문 (Q&A)

Q. 왜 드리프트가 오면 처음부터 다시 학습하지 않고 fine-tuning하나요?

A. 최근 1개월(21거래일)로 만들 수 있는 학습 시퀀스는 20개 남짓이라, 약 1.6만 개 파라미터의 LSTM을 처음부터 학습시키기엔 너무 적어 결과가 불안정합니다. 이미 3년치로 학습된 Production 가중치에서 낮은 학습률(1e-4)로 10 epoch만 이어서 학습하면, 기존 지식을 유지하면서 최근 변화만 반영할 수 있습니다.

Q. fine-tuning 후 게이트 RMSE는 믿을 만한가요?

A. 좋은 질문입니다. fine-tuning에 쓰는 41행으로 만든 21개 시퀀스를 다시 8:2로 나누면 검증 샘플은 약 5개뿐입니다. 표본이 적어 RMSE가 우연히 낮게 나올 수 있다는 한계가 있습니다. 서브노트나 조별 운영 설계에서 "재학습 검증셋을 어떻게 확보할 것인가"를 다루면 높은 평가를 받을 수 있습니다.

Q. 스케일러는 왜 다시 fit하지 않나요? 데이터가 바뀌었는데요.

A. 모델 가중치는 Day1에 fit한 min-max 기준으로 학습되어 있습니다. 재학습 때 스케일러를 새로 fit하면 같은 가격이 다른 숫자로 들어가 기존 가중치와 어긋나고, fine-tuning 자체가 의미를 잃습니다. 학습 시점과 서빙 시점의 전처리를 한 곳(data/features.py)에서 관리하는 이유도 같습니다.

Q. Lazy와 Eager 중 무엇이 정답인가요?

A. 정답은 없습니다. Eager는 시작이 느리지만 첫 사용자도 빠른 응답을 받고, 모델 파일 문제를 기동 시점에 바로 발견합니다(컨테이너 배포에서 Eager를 쓰는 이유). Lazy는 시작이 빠르지만 첫 요청이 느리고 오류가 늦게 드러납니다. 측정한 수치를 근거로 서비스 상황에 맞는 선택을 설명하는 것이 이 실습의 목표입니다.

Q. RMSE 말고 다른 드리프트 감지 방법은 없나요?

A. 이번 실습은 실제값(actual)이 곧바로 들어오는 시뮬레이션이라 예측 오차(RMSE)로 판정합니다. 실무에서는 정답이 늦게 도착하는 경우가 많아, 입력 분포 자체의 변화를 KS 검정이나 PSI로 먼저 감지하기도 합니다. 조별 미니 프로젝트의 확장 과제로 시도해 볼 만합니다.

부록 3. Day2 Dockerfile 한 줄씩 읽기

Docker 기초 개념(이미지, 컨테이너, 격리, 포트 매핑)은 선행 과목에서 다룬 내용을 참고하고, 여기서는 이번 실습의 Dockerfile이 **무엇을 왜 하는지**만 짚습니다.

Dockerfile 구문	의미	이번 실습에서의 포인트
FROM python:3.11-slim	베이스 이미지	로컬 가상환경과 같은 Python 버전으로 환경 일관성 확보
WORKDIR /app	컨테이너 내부 작업 경로	호스트 폴더와 무관. 이후 모든 상대 경로의 기준(프로젝트 루트 역할)
COPY requirements.txt . / RUN pip install ...	의존성 먼저 설치	소스보다 먼저 복사해 레이어 캐시 활용 (소스만 바뀌면 재설치 생략)
COPY serving_app/ data/ scripts/	소스와 데이터 복사	빌드 컨텍스트가 프로젝트 루트여야 함 (compose의 `context: ..`)
RUN ... train_baseline_v1.py && train_and_register.py	빌드 시점에 시드 데이터로 학습·등록	MLflow(sqlite)와 서빙이 같은 컨테이너 파일시스템에 있는 자기 완결형 이미지
EXPOSE 8000 / ports: 8000:8000	컨테이너 포트 공개와 호스트 연결	로컬 실습 포트도 8000으로 통일하면 혼란이 없음
ENV MODEL_SOURCE=mlflow, LOADING_MODE=eager	런타임 설정	Day1~2의 환경변수 전환 구조를 그대로 컨테이너에 적용
CMD ["uvicorn", ...]	컨테이너 시작 명령	로컬에서 실행하던 uvicorn 명령과 동일